

Three ways to search binary embeddings

A guide to the algorithms, their costs, and the evidence

11 September 2026

1 What are we trying to make faster?

I started this project after reading Exa’s description of its vector database [1]. If we look at the work inside a selected cluster, one thing we can explore is whether the query can help us choose documents before we score them. We can always calculate a score for every document and return the best ones, but that gets expensive as the collection grows. Skipping documents might help, so long as the work of choosing them costs less than the scores we avoid, and we still find the results we wanted.

We’ll compare three ways to search the same binary document embeddings. In method A, we open selected clusters and scan their documents. In B, we open clusters and then split the documents using bitplanes. In C, we open clusters and look up bit patterns, starting with the pattern the query prefers. We’ll call C *weighted key lookup*; this is the concrete version of starting at the ideal answer and “walking backwards.”

If we fix the document count, RAM budget, and required recall, which method gives us the lowest latency? Recall tells us how many of the correct reference results we recover, so we need to keep it in the comparison as we change the search. And if we want to estimate latency, counting calculations is only part of the job: we also need to look at the memory those calculations read. Avoiding a document score can introduce work elsewhere.

The experiments give us three different outcomes to explain. On the tested real embeddings, neither alternative established a speed advantage over a well-tuned scan. With constructed queries whose important coordinates stayed in one prefix, key lookup had a small advantage over a closely related directly routed scan. When the important coordinates changed with each query, bitplane branching had a large advantage in the constructed case. So the useful question is what makes those cases different, rather than which method wins everywhere.

Let’s start with one query and six documents, follow it through each method, and then count the work and storage. From there, we can look at recall, choose parameters, and compare the measurements. At the end we’ll use the calculations to consider a billion documents. Those larger latency numbers are forecasts; the complete native indexes were tested through eight million documents.

1.1 One query, six documents, one score

Consider an embedding: a vector of numbers produced from text. We’ll keep the query as numbers, but store just the sign of each document coordinate. A stored bit of 1 means +1, and a bit of 0 means −1; this is *binary quantization*. For our example, let

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

The score is the dot product, so we multiply corresponding coordinates and add the results. If we choose signs 11001, every contribution is positive: that is the document pattern the query prefers, and it scores $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$.

Document ID	Stored signs	First-bit cluster	Full score
0	01001	0	1.1
1	00100	0	-1.3
2	10100	1	0.1
3	11001	1	2.5
4	11000	1	1.9
5	00011	0	-1.1

Here the correct top two are documents 3 and 4. If scores tie, we put the smaller document ID first, and then use that same rule when checking every method. We can also run this example against the native implementations with `examples/small_search.py`.

If we look at coordinate 1, a matching sign contributes $+0.7$, while a mismatch contributes -0.7 . That one change lowers the score by 1.4. At coordinate 5, the loss is only 0.6, so the query magnitude tells us how expensive a wrong sign would be. We can use that observation to guide the search. For a query with d coordinates, write its ideal score as $Q = \sum_{i=1}^d |q_i|$. Writing b_i for the decoded sign ($+1$ or -1), if the mismatching coordinates have total magnitude p , then

$$S(q, b) = Q - 2p, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

For document 4, only coordinate 5 mismatches, giving $S = 2.5 - 2(0.3) = 1.9$. From here, ranking by the largest score or the smallest mismatch penalty gives the same order. Ordinary Hamming distance counts each mismatch equally; here we use the query magnitudes as weights.

1.2 From text to an index, then back to results

Suppose the collection contains N documents. Before any queries arrive, we embed the text, choose a representation width d , pack the document signs, and build the index. We also assign each document to one cluster. A cluster is a stored subset of the collection, and we can form those subsets by learning from the vectors or by using a fixed sign prefix.

Matryoshka training makes selected prefixes useful at smaller dimensions [2], which is why a shorter representation is worth considering. But it doesn't sort each query's coordinates by magnitude, or guarantee that 24 sign bits preserve its nearest neighbors. If we truncate an embedding and then take its signs, we have made two separate changes to the representation. We'll hold that representation fixed across methods in the primary comparison.

Once a query arrives, we embed and normalize it, select clusters, and search their documents. Opening one cluster is one *probe*. With C clusters and P probes, balanced clusters would give us about NP/C selected documents. If the clusters aren't balanced, we need to add their actual sizes instead. And if we want to search the whole collection, we can set $C = P = 1$.

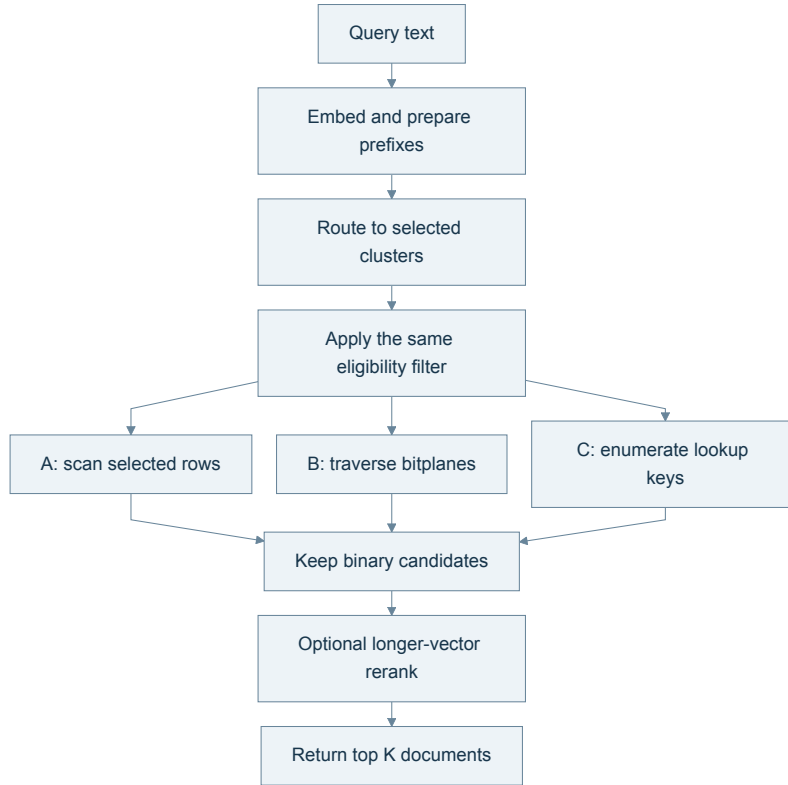


Figure 1: The common query flow. The three local search methods share the same query and selected clusters when we isolate their work. Longer-vector reranking is optional; it is excluded from the primary binary-score comparison.

A production pipeline can also filter documents or rerank candidates with longer vectors or another model. If those stages are enabled, they belong in the complete request measurement. For the primary index comparison here, we use cached query embeddings, the same binary score, and no float reranker. Later we’ll measure actual text queries so we can see the complete wait.

Exa’s published vector-database description is the starting point for this cluster-and-scan design and compact scoring [1]. Our A follows that broad structure locally; the timings are not measurements of Exa’s production system. We also need to distinguish two routing choices we’ll encounter: a sign prefix and learned IVF clusters. A k -bit prefix defines up to 2^k patterns, not k clusters.

2 Three ways to search the selected documents

2.1 A: cluster, then full scan

Let’s first scan the example so we have a result we can check. Coordinate 1 defines our two clusters; opening cluster 1 and then cluster 0 gives us documents 2, 3, 4, followed by 0, 1, 5. We calculate each full score and keep the best two. If we visit the documents in another order, the final ranking is still the same.

We can keep those results in a *top-K heap*: it holds the best K found so far, with the worst retained result ready to replace. Follow the example with $K = 2$. After documents 2 and

3, the retained scores are 0.1 and 2.5; when document 4 scores 1.9, it replaces document 2. The remaining documents don't improve that pair, so we finish with the reference answer.

Listing 1: Full scan. Every selected document receives a full score.

```
scan(query, selected_clusters, K):
    table = prepare_score_table(query)
    best = empty_top_k(K)
    for cluster in selected_clusters:
        for document in cluster.documents:
            score = full_score(table, document.code)
            best.offer(score, document.id)
    return best.sorted_by_score_then_id()
```

If we inspect `full_score`, we don't need a separate multiplication for every sign. For each group of four coordinates, we can precompute the contributions of all 16 possible sign patterns and put them in a score table. A 256-bit document then needs 64 table terms. There is still work in a lookup: read the bits, extract the pattern, load its value, and add it to the score. Counting one table term therefore isn't counting one CPU instruction.

With g coordinates per group, the same idea gives $G = \lceil d/g \rceil$ groups and $G2^g$ stored table values, allowing padding in the last group. We prepare the table for each query, and all three methods use it. If we increase g , we trade a larger table for fewer terms per document. The measured implementation uses $g = 4$.

Let $\mathcal{P}$ be the selected clusters and n_c the size of cluster c . A scores

$$F_A = \sum_{c \in \mathcal{P}} n_c$$

documents and evaluates GF_A table terms. A useful local time model is

$$T_A^{\text{local}} \approx T_{\text{table}} + F_A c_{\text{scan}} + T_{\text{select}}(F_A, K). \quad (2)$$

Here c_{scan} is the time spent reading and scoring one contiguous document at the chosen width. We have written selection separately so its cost stays visible. If we use a measured per-row coefficient that already includes selection, we shouldn't add that cost again. The heap has a worst-case bound of $O(F_A \log K)$, although many scores leave it unchanged.

2.2 The scan's storage and temporary memory

Now let's look at what we store. If a packed word has w bits, it takes $u = w/8$ bytes, and a code needs $u \lceil d/w \rceil$ bytes. We also need the document ID; call its width b_{id} . Adding those two parts for every document gives

$$M_{\text{rows}} = N(u \lceil d/w \rceil + b_{\text{id}}). \quad (3)$$

For $d = 256$, $w = 64$, and eight-byte IDs, that is $32 + 8 = 40$ bytes per row, or 40 MB for one million rows in decimal units. We still need to account for cluster directories, spare vector capacity, and allocator bookkeeping; the row payload doesn't include them.

Routing adds storage and query work too. For example, C float centroids of width d_r need $4Cd_r$ bytes for their coordinates. If we compare every centroid exactly, each query

examines Cd_r coordinates and then chooses the best P centroids. A sign-prefix router does different work, so we can't give routing one price without specifying how it is implemented.

During the query, we also keep the query vector, score table, selected cluster IDs, and current results in memory. Call this shared temporary allocation Q_0 ; its main terms scale as $O(d + G2^g + P + K)$. If Python and native result arrays coexist, both need to be counted. We can then add Q_0 to the stored rows, routing, and container allocation to estimate A's serving memory. Building the index can peak higher because the input arrays and newly arranged index exist together. This is why file size, logical payload, and process RAM need separate counts.

Also, if we look at how the CPU reads the scan's data, the codes in a cluster sit consecutively. The CPU can fetch future cache lines while working on the current ones, and SIMD instructions can process several values together. Neither changes the number of documents we score, but both can change the time per row. So even before introducing another method, we have a reason to be careful about predicting latency from arithmetic alone.

2.3 B: arrange the same bits by coordinate

The query tells us which sign it prefers at each coordinate. One thing we can try is selecting documents through that sign before calculating their full scores. For this, B stores a *bitplane*: one coordinate across all documents in a cluster, with the sign data arranged as columns. We also keep the packed document rows, because selected candidates still need the same full score as A.

Consider cluster 1 in the example, where the local document order is [2, 3, 4] and the codes are 10100, 11001, and 11000. If we read coordinate 2 down those rows, we get plane 011: documents 3 and 4 have that bit set. Coordinate 3 gives 100. The query prefers 1 at coordinate 2 and 0 at coordinate 3, so either selection keeps documents 3 and 4.

We can record the remaining local positions in a candidate mask. Starting with parent mask A and plane B_i , a positive-query split keeps the matching positions in one child and the others in another:

$$A_{\text{match}} = A \& B_i, \quad A_{\text{other}} = A \& \neg B_i.$$

If the query coordinate is negative, we swap which child is the match. Both results are intersected with the parent, so padding positions can't become real documents.

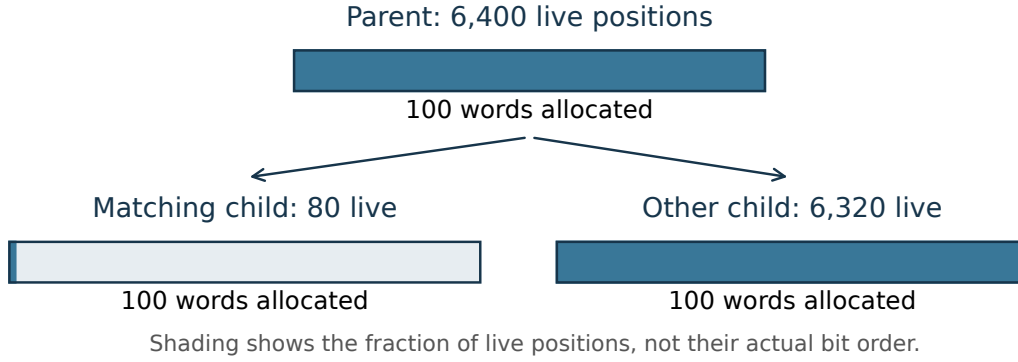


Figure 2: Membership shrinks, but a dense mask keeps its physical width. Each child still has 6,400 positions, or 100 words. This larger illustration counts storage; the six-document example explains the returned IDs.

2.4 Follow the promising branch, keep the alternative

If we order the query coordinates by magnitude, we get 1, 4, 2, 3, 5. In cluster 1, coordinate 1 is already fixed, and all three documents match the query at coordinate 4. The split on coordinate 2 then gives [3, 4] in one child and [2] in the other. Matching adds no penalty, while the other child adds 0.5, so we try the matching child first.

Trying that child first doesn’t mean we can discard the other one. A document could match this bit and then disagree on several later bits. Instead, we keep unexplored children in a queue, ordered by a lower bound on mismatch penalty. Equation 1 gives a node with known penalty p a score upper bound of $Q - 2p$, allowing every unexamined coordinate its best sign. If even that upper bound falls strictly below the worst retained top- K score, the node can’t improve the result. Equal-score possibilities still need the ID tie rule, and the native check stays conservative around floating-point comparisons.

Listing 2: Bitplane search. The queue preserves alternatives to the preferred path.

```
branch(query, clusters, K, leaf_size, node_budget):
    table = prepare_score_table(query)
    order = coordinates_by_decreasing_abs(query)
    queue = one full candidate mask per selected cluster
    best = empty_top_k(K)
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): return best
        if node.count <= leaf_size or no_bits_remain(node):
            score_each_set_document(table, node.mask, best)
        else:
            matching, other = split_on_next_bit(node, order)
            queue_nonempty_children(queue, matching, other)
    return best.sorted_by_score_then_id()
```

The listing gives us the search policy; counters, ownership, and numerical tolerance live in the native source. Once a node is small enough, we treat it as a leaf and score its

documents directly. The leaf size L tells us when to make that switch. A node budget gives us another way to stop, but it can end the search before the bounds establish the answer. With no work limit and safe bounds, we can finish exactly within the selected clusters. We still haven't made any guarantee about clusters we didn't probe.

We can also occasionally choose another queued node instead of the one with the best bound, using a probability parameter. This changes the exploration order, not the score. Under a fixed budget it could help or hurt recall, so it needs seeded tests rather than an assumed gain. The tested real-data settings didn't show a consistent benefit from random exploration.

2.5 Count the full-width work

For cluster c , each dense mask spans $W_c = \lceil n_c/w \rceil$ words. Let J_c count its splits and E_c the leaf masks whose set bits are enumerated. The full-width visits are

$$V_{\text{split}} = \sum_{c \in \mathcal{P}} J_c W_c, \quad V_{\text{leaf}} = \sum_{c \in \mathcal{P}} E_c W_c. \quad (4)$$

Even at a leaf, the loop checks zero words to find out that they are empty, and then the set bits tell us which documents to score. If we count only the surviving documents, we miss that first part of the work. An early cost estimate was too optimistic for exactly this reason: it omitted the leaf-word visits.

Let's follow the larger mask example. We start with 6,400 documents and narrow the candidates to 80, but the mask still contains 100 words. If we make ten more dense splits, those loops visit 1,000 words even if only a few bits remain set. Each visit reads the parent and plane, constructs children, and writes results; we also spend time allocating masks and maintaining the queue. These costs remain even when the candidate count looks small.

If F_B documents are eventually scored, a practical decomposition is

$$\begin{aligned} T_B^{\text{local}} \approx & T_{\text{table}} + T_{\text{order}} + V_{\text{split}} c_{\text{split}} + V_{\text{leaf}} c_{\text{leaf}} \\ & + F_B c_{\text{gather}} + T_{\text{queue}} + T_{\text{select}}(F_B, K). \end{aligned} \quad (5)$$

To use this equation, we need to know what each measured price includes. The split price should cover the chosen mask allocation policy, and scattered row reads may cost more than contiguous scan reads. If a leaf timer already covers finding set bits, fetching the rows, and scoring them, we can use that combined measurement. Adding those same stages separately would count them twice.

B's additional stored plane payload is

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil. \quad (6)$$

For one large cluster, this is approximately $Nd/8$ bytes. If we use many small clusters, padding adds more. We also need to count the masks waiting during a query: if $A_c(t)$ masks for cluster c coexist at time t , their payload peaks at $u \max_t \sum_c A_c(t) W_c$. Queue-node allocation and shared scratch come on top of that. Counting only the mask we're visiting would leave out the alternatives still in the queue.

2.6 C: begin with the query's key

There's another way to use the query's preferred signs. Instead of choosing a sequence of columns as B does, we can build a directory over a fixed set of h coordinates. C stores documents with the same short key together within each cluster, and the directory gives the start position and count of each occupied key. The document IDs already identify the posting range, so this implementation doesn't add a second ID array for postings.

Let's use coordinates 2 and 3 as the key in the same example. The query wants 10. If we flip coordinate 3, the penalty is 0.4; flipping coordinate 2 costs 0.5, and flipping both costs 0.9. That gives us the weighted order 10, 11, 00, 01 to try in the directory.

Query key: coordinates 2 and 3 = 10

Key order	Known penalty	Cluster 0 IDs	Cluster 1 IDs
10	0.0	0	3, 4
11	0.4	empty	empty
00	0.5	5	empty
01	0.9	1	2

Figure 3: One shared key sequence is looked up in both selected clusters. The ideal key already finds the top two, but that fact alone is not a safe stopping rule. Empty lookups still cost work.

The first key returns document 0 from cluster 0 and documents 3 and 4 from cluster 1. We score them fully and get 1.1, 2.5, and 1.9, leaving [3, 4] as the best pair. Can the next key improve it? Its known penalty is 0.4, so even its full-score upper bound is only $2.5 - 2(0.4) = 1.7$, below 1.9. Here that bound lets us stop before the empty 11 lookups. Figure 3 includes the wider sequence to show the directory and its empty keys; a walk without the bound or a work limit would inspect them too.

It would be tempting to stop as soon as we find K documents, but the short key hasn't checked the other coordinates. A matching document might score poorly on them, while another key contains a better full score. We therefore score the full code and keep an upper bound for unvisited keys, rather than treating the candidate count as proof that we've finished.

Listing 3: Weighted key lookup. Generate the key sequence once; charge every cluster lookup.

```

keys(query, clusters, K, candidate_target, lookup_limit):
    table = prepare_score_table(query)
    ideal = signs_of_fixed_key_coordinates(query)
    queue = [empty_flip_set_with_zero_penalty]
    best = empty_top_k(K)
    while queue is not empty:
        if remaining_bound_cannot_improve(queue, query, best): break
        state = remove_lowest_penalty(queue)
        key = ideal XOR state.flip_mask
        for cluster in clusters:
            if lookup_limit_is_reached(): return best
            count_one_lookup_including_misses()
            for document in cluster.directory.get(key, []):
                best.offer(full_score(table, document.code), document.id)
        if candidate_target_is_reached(): return best
        add_next_unique_flip_sets(queue, state)
    return best.sorted_by_score_then_id()

```

If we follow the queue, it tries combinations in increasing total mismatch weight. That is different from simply increasing the number of flipped bits: two low-weight flips may cost less than one high-weight flip. The native subset-generation rule avoids generating the same combination repeatedly. For an unvisited key, $Q - 2p_{\text{key}}$ remains a safe full-score upper bound because all omitted coordinates could match, although that allowance can make it loose.

We can limit the work with a positive candidate target or lookup limit; zero disables a limit in the native API. Either limit can stop early and lose recall. The target is checked after a key has been processed across all selected clusters, while the lookup limit can stop during that loop. To finish exactly within the selected clusters, we must instead exhaust potentially improving keys or prove that none remain, with score ties handled consistently.

2.7 What the key directory costs

Let H count actual directory lookups and F_C the full scores. With E generated keys and no limit cutting a cluster loop, $H = PE$. An empty key contributes to H but not F_C . A time model for the implemented policy is

$$\begin{aligned}
 T_C^{\text{local}} \approx & T_{\text{table}} + T_{\text{key order}} + T_{\text{key queue}} + Hc_{\text{lookup}} \\
 & + F_C c_{\text{posting}} + T_{\text{select}}(F_C, K).
 \end{aligned} \tag{7}$$

Rows within a posting range are contiguous, but reading many short ranges can still cost something different from one long scan. The queue cost also depends on how many generated states remain live. Since this implementation shares one stream across clusters, we charge that queue once, not P times.

For U_c occupied keys in cluster c , the logical directory payload is $\sum_c U_c(b_k + 2b_o)$: a key, start, and count. The current key field uses four bytes and each range field uses eight. Allocated memory is larger. The C++ directory uses separate chaining, with one node per occupied entry and a bucket-pointer array. If B_c is the number of hash buckets,

its allocation includes $\sum_c (U_c b_{\text{node}} + B_c b_{\text{ptr}})$, plus container and allocator overhead. The 20-byte field sum is not an allocated-node measurement.

If we widen the key, posting ranges may get shorter, but useful keys may also take more work to reach. A very short key has the opposite problem: it finds documents easily while leaving nearly all the scoring work. So finding a nonempty key isn't the same as finding the best documents. And if we want to relax a pattern to something like $1*0$, we need a different operation; an exact-key hash table doesn't answer that wildcard request in one lookup.

3 Recall and the choice of parameters

3.1 Recovering the reference results

We now have ways to avoid some scores, but how do we check whether we kept the right documents? Let $G_K(q)$ be the reference top- K IDs from a complete scan, and $A_K(q)$ the IDs returned by the method we're testing. For $K \leq N$, recall for query q counts the reference IDs we recover:

$$R_K(q) = \frac{|G_K(q) \cap A_K(q)|}{K}. \quad (8)$$

In our six-document example, $G_2 = [3, 4]$. If we return $[3, 0]$, we recover one of the two IDs, so recall is 50%; returning $[3, 4]$ gives 100%. A large candidate set doesn't settle this question by itself. It has to contain the reference IDs, and the final selection has to keep them.

We also need to be clear about which reference we're using. The main experiments compare against the full binary-document score in Equation 1. If we instead used a float-vector reference, we would also measure the loss from quantization. Human relevance labels ask another question: whether the text is useful for the query. Recovering the binary neighbors doesn't by itself establish either relevance or accurate language-model answers.

If we follow a reference document through the pipeline, we can lose it when its cluster isn't opened, or when local search misses it inside an opened cluster. Fix a query and choose one of its K reference IDs uniformly. Let V mean that we visit its cluster, and L mean that the local method returns it. Then

$$R_K(q) = \Pr(V \mid q) \Pr(L \mid V, q). \quad (9)$$

This uses conditional probability; it doesn't assume the two events are independent. And if we average over queries, we generally can't multiply the two separately averaged probabilities instead. A's local scan has conditional survival one, but routing can still remove a result. B and C share that routing limit, with any local work limits adding another way to miss it.

3.2 Derive one useful distributional model

To estimate recall mathematically, we need assumptions about both the document signs and the query weights. Let's start with independent document signs, each equally likely

to be 0 or 1. Give m strong query coordinates magnitude a , and the remaining $d - m$ coordinates magnitude ϵ . For this example, require

$$a > (d - m)\epsilon. \quad (10)$$

Here one strong mismatch costs more than all possible weak mismatches together. So a document matching all the strong coordinates outranks every document that mismatches one, regardless of what happens on the weak coordinates. We've deliberately constructed this condition; Matryoshka training alone doesn't give us that guarantee.

If the signs are independent and balanced, a document matches those m signs with probability $p = 2^{-m}$. We can then count how many matches occur among N rows. Call that count Z ; it follows a binomial distribution:

$$\Pr(Z = z) = \binom{N}{z} p^z (1 - p)^{N-z}, \quad \mathbb{E}[Z] = Np.$$

Suppose we open only this ideal strong-sign group and score all its rows. If $Z \geq K$, the whole top- K is inside it. If $Z < K$, we recover Z of those results. The recall is therefore $\min(Z, K)/K$, and averaging over the possible counts gives

$$\mathbb{E}[R_K] = \frac{1}{K} \sum_{z=0}^N \min(z, K) \binom{N}{z} p^z (1 - p)^{N-z} = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i). \quad (11)$$

We can read the second form as adding the probabilities of recovering at least one result, at least two, and so on through K . That lets us avoid a sum over every possible corpus count.

Consider $N = 2$, $m = 1$, and $K = 1$. The ideal group is empty with probability $1/4$, contains one document with probability $1/2$, and contains two with probability $1/4$. We can calculate expected recall directly: $0(1/4) + 1(1/2) + 1(1/4) = 3/4$. Notice that the average group size is one, but some groups are empty. If we substituted that average as though it were a guaranteed count, we would incorrectly predict 100% recall.

Ideal-group size Z	Probability	Recall@1
0 matches	1/4	0
1 match	1/2	1
2 matches	1/4	1

Figure 4: Two independent documents and one required matching bit. Opening only the matching group succeeds with probability $3/4$. The empty-group outcome explains why the mean group size is insufficient.

At $N = 10^6$ and $m = 13$, the ideal group averages about 122 rows. With $K = 100$, some groups can still be too small, so a complete B or C search may need to continue into other groups. Equation 11 describes stopping after the ideal group; it doesn't describe every search policy. If we increase a work budget, we have to consider which additional groups we visit and the scores and sizes of those groups.

For ordinary queries, taking a fraction of the coordinates doesn't mean we keep the same fraction of useful ranking information. Correlated bits, different query magnitudes, and approximate routing all affect which neighbors survive. We can try a normal approximation to the prefix and full scores, but then we need to calibrate it and check it independently. In the earlier local Nomic check, recall errors reached about 9.6 percentage points, missing the proposed two-point criterion. So that approximation cannot support a claim about real-data recall at a billion rows here.

3.3 Which parameters can we change?

Some choices change what we build: the code width d , cluster count C , routing method, and C 's key coordinates and width h . Others can change from query to query: probes P , B 's leaf size and node budget, C 's lookup and candidate limits, and the requested K . If we change d , the reference ranking changes too, so we need to compare all methods again using that same representation.

We can use the cost equation to narrow the settings worth trying. Suppose B follows one promising path, each split halves the candidates, and the masks keep width $W = \lceil N/w \rceil$. After j splits, we expect $N2^{-j}$ rows to reach the scorer. If we temporarily leave out changes in queue cost, this gives

$$T_B(j) \approx T_0 + jWc_{\text{split}} + Wc_{\text{leaf}} + N2^{-j}c_{\text{score}}. \quad (12)$$

If we make one more split, we save about $N2^{-(j+1)}$ scores and pay for another full-width split. It helps in this model when the saved scoring time exceeds Wc_{split} and the additional queue cost. For example, with $N = 6,400$ and $j = 5$, another balanced cut saves about 100 scores. We compare the time of those 100 scores with the time of a 100-word split; the two counts alone aren't comparable.

Treating j as continuous gives a starting point:

$$\frac{dT_B}{dj} = Wc_{\text{split}} - N \ln(2)2^{-j}c_{\text{score}} = 0, \quad j^* = \log_2 \left(\frac{N \ln(2)c_{\text{score}}}{Wc_{\text{split}}} \right).$$

This gives us a place to start, and then we check the neighboring valid integer depths, the score assumptions, and the required recall. The calculation assumes one useful balanced path; it doesn't cover arbitrary branch exploration. Also, since W grows with N , a larger collection doesn't automatically call for a larger optimal depth. The dense masks themselves still take more work as the collection grows.

The same distribution gives us a reason to try a different leaf size. At one million rows, the 13-bit ideal group has mean about 122 and standard deviation about 11. If we stop at 128, some slightly larger groups force another split on a weak coordinate. The rough margin $122 + 3(11) \approx 155$ suggested testing $L = 160$. Since the preceding 12-bit group averages about 244 rows, it will usually still split. We checked this suggestion experimentally; the approximation alone doesn't guarantee that it is the right threshold.

The complete choice for method a is

$$\min_{\theta \in \Theta_a} T_a(\theta) \quad \text{subject to} \quad R_a(\theta) \geq r, \quad M_a(\theta) \leq M_{\text{limit}}. \quad (13)$$

Here the vector θ holds the settings available to a method. A derivative is useful when we know the relevant functions, but N and RAM alone don't tell us cluster occupancy

or routing recall. We therefore measure promising settings, keep the feasible ones, and compare the best *tested* choices. That includes giving A enough routing options and letting B stop splitting when a scan-like leaf is cheaper.

4 What the experiments establish

The work formulas tell us what each method must do, but we need measurements to find out whether the scoring it avoids pays for the search it adds. We compare real embeddings first, then use constructed inputs to isolate how much the query’s strong coordinates matter. The saved source record identifies the runs behind these comparisons [5].

4.1 Make the comparison fair

A, B and C receive the same documents and floating-point queries, use the same native weighted binary scorer, and return the top 100 with score ties broken by document ID. When we measure recall here, we measure recovery of that exact binary ranking; these comparisons use neither a float reranker nor human relevance judgments.

The machine is an Apple M5 Max with 128 GiB of physical memory, while each worker runs under a separate 32 GB decimal memory budget using one CPU thread and one query at a time. A fresh process builds a configuration, warms it, and then measures routing plus search. The outer timer includes the call from Python into C++, but cached-vector measurements exclude text encoding. Repeating whole process blocks helps expose timing variation; the recorded power settings do not prove that CPU frequency remained fixed.

We tune parameters on one set of query IDs, freeze the choices, and evaluate them on different IDs, allowing each method its own fastest tested setting that meets the recall requirement. For A, the tested options include intermediate probe counts and direct routing by sign keys. For B, they include large leaves, so it can score a selected cluster without unnecessary splits. If we left these options out, a poor baseline or a forced traversal could decide the result before the timing begins.

The real study retains two ways to select settings: one accepts a setting when its average tuning recall reaches the target, while the conservative rule requires a lower estimate from repeated resampling of tuning queries to reach it. The conservative rule leaves more margin, but cannot promise recall on future queries. To establish a speedup, both methods must also meet the target and the paired 95% interval for their time ratio must lie above one. Queries and whole process blocks are resampled together; timing repeats do not count as additional independent queries.

4.2 Real embeddings did not establish an alternative speedup

For the real study, the documents are 256-bit codes from Nomic embeddings of MS MARCO passages [4, 3]. After combining 6,821 tuning settings, the frozen choices were evaluated on 200 query IDs excluded from the earlier 300-query collection. On this sample, all 36 conservative target points met their requested recall, compared with only 7 of 36 points chosen by average tuning recall.

In the conservative results at one million documents below, each row sets a common minimum recall requirement and shows each method’s median routing-plus-search time. If we compare methods within a row, their achieved recall also happens to be equal.

Target	A (ms)	B (ms)	C (ms)	Recall (%)
80%	0.675	0.699	0.751	82.53
90%	1.634	1.686	1.880	91.73
95%	3.360	3.430	3.706	95.47
99%	9.580	9.965	11.013	99.25

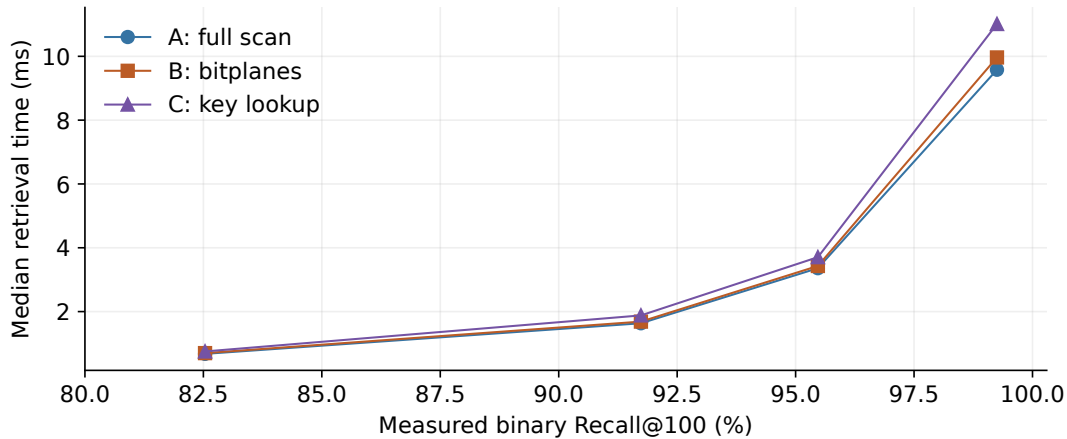


Figure 5: Four frozen recall-target choices on one million real passage embeddings and 200 independent query IDs. Lines connect the tested settings for readability; they do not predict intermediate settings. At each shown recall, A has the smallest median.

Across the real study, none of the 48 evaluated comparisons established a B or C speedup under the stated rule. Although fewer documents received full scores, that saving did not produce a supported reduction in complete retrieval time. This conclusion applies to this data, score, implementation and tested parameter range.

A later control gives C a one-bit key, A’s routing layout, and unlimited candidate and lookup budgets, while A and B keep their frozen settings. This control uses 100 further query IDs, excluding all 500 previously used IDs. A, B and C measured 9.541, 9.855 and 10.545 ms, respectively, and all three achieved 98.95% recall, below the requested 99%. That remains a miss; we do not round it into a pass. The sample is separate from the earlier successful 200-query evaluation, and no parameter was changed in response to its outcome.

4.3 Constructed data shows when skipping scores can help

With constructed inputs, we can isolate the effect of keeping the strong coordinates fixed or changing them with each query. Document signs are independent and balanced across 256 coordinates. Each query has 13 strong coordinates of magnitude one, with magnitude 0.0001 on the remaining coordinates. A document matching every strong sign therefore beats any that mismatches one, regardless of the weak signs. The fixed case keeps the

strong positions the same; in the changing case, each query chooses different strong positions uniformly without replacement. These vectors are constructed, not measured text embeddings.

The final follow-up uses 40 tuning queries and 20 separate evaluation queries, with three process blocks and three timing repeats. It adds neighboring key widths, cluster counts and leaf sizes, and then remeasures earlier choices. At one million documents and the 99% requirement, the results below show that all methods reach 100% recall except A in the changing case, at 99.95%.

Strong positions	A (ms)	B (ms)	C (ms)
Fixed	0.0140	0.0160	0.0128
Changing	24.9235	0.1446	14.6301

With fixed positions, A selects two of 16,384 clusters by direct routing, while C uses one physical cluster and a 13-bit key. C’s paired speedup estimate is 1.098, with a 95% interval of 1.066–1.147, and the time difference is about 1.25 microseconds. Both methods select documents through the same strong signs, but C combines key enumeration and scoring in one native call, whereas A routes and scans through separate calls. The small measured advantage therefore concerns this implementation of closely related candidate selection.

If the strong positions change, a fixed key cannot consistently select them. B can follow them because it orders coordinates using each query’s magnitudes; its selected setting uses one cluster, unlimited traversal and a leaf size of 160. A opens 255 of 256 centroid clusters, and B’s paired speedup estimate is 172.3, with a 95% interval of 169.1–177.2. C uses a 14-bit key and has a lower median than A, but its paired interval against A includes one, so that median alone does not establish an advantage.

We can see the reason for the leaf choice in the occupancy calculation: at one million rows, the ideal 13-bit group has mean size about 122 and standard deviation about 11. A threshold of 128 can force another split on a weak coordinate, whereas a threshold of 160 leaves more room to score that group. The experiment checks this suggestion from the distribution; it does not establish that 160 is best for arbitrary embeddings, sizes or machines.

4.4 Check the answer, the work, then the predicted time

Before checking work or timing predictions, we need an independently calculated answer. The checks calculate scores directly from document signs and sort by score and ID; larger runs count strong and weak mismatches from packed bits to avoid a huge expanded matrix. Every complete native call in the scaling check returned the independent reference ranking.

To check the predicted work, we compare it with native counters. Across the earlier controlled tuning and evaluation runs, 447 applicable per-query predictions matched exactly, including full-width split and leaf visits, scored rows, key attempts including empty lookups, and tracked mask peaks where applicable. Queries outside the formulas’ stated conditions remain in the measurements, but do not count as successful predictions.

Even with correct counts, we cannot assign every row the same time cost. In a 16-million-document buffer, scoring about 1,958 scattered rows took 0.159 ms with reused candidates

and 0.455 ms with changing candidates. Candidate generation stayed outside the timer, but still changed which memory had been accessed recently. Fits trained on one- and four-million-row buffers underestimated a 16-million-row check by as much as 45.2% for reused candidates and 27.6% for changing candidates, and both failed fits remain saved. The largest row store was about 640 MB, so these checks do not validate access costs for a 40 GB store.

The later complete-call model uses one latency median per corpus size, correcting an earlier fit that gave fast individual queries too much weight when predicting a median. The revised model also accounts for result-selection work in C’s small postings. For B, the leaf threshold grows as $L = 160N/10^6$, so a larger corpus keeps the intended strong-coordinate depth instead of forcing more cuts.

Because the revised four-million-row check had already been inspected, it serves as a diagnostic check. The coefficients were frozen before the separate eight-million-row run below, where a positive error means the prediction was slower than the measurement.

Positions	Method	Measured (ms)	Predicted (ms)	Error
Fixed	A	197.029	203.016	+3.0%
Fixed	B	0.854	0.884	+3.6%
Fixed	C	0.0481	0.0518	+7.7%
Changing	A	196.664	197.816	+0.6%
Changing	B	0.900	0.859	−4.5%
Changing	C	202.767	207.086	+2.1%

All six checks met the earlier 20% criterion, testing transfer to a larger corpus under the same prescribed workload and existing query IDs. These model-check configurations are separate from the tuned comparisons in the preceding table. Although the largest observed error was 7.7%, that does not give us a guaranteed bound for future queries or a billion documents.

4.5 Measure actual text requests

When a user supplies text, retrieval covers only part of the wait. The final check therefore runs 20 existing independent-query texts through the cached local Nomic model on MPS, normalizes each vector, routes it and calls native search, with a batch size of one. Three blocks and two repeats give 120 requests per method, or 360 requests in total. Each sequence shape is warmed, and the request timer excludes model loading, index building and reference creation.

Using the earlier frozen settings, all three methods achieve 99.45% binary recall on this sample. The table gives the stage medians alongside the separately measured request total.

	Encode (ms)	Retrieve (ms)	Total (ms)	Total p95 (ms)
A	6.488	9.406	16.313	19.201
B	6.500	9.651	16.303	19.217
C	7.543	11.201	18.912	21.898

The median is the middle time, and p95 is the 95th-percentile time. The tiny difference between the A/B totals does not establish a speedup. If we add the stage medians, we need not get the total median, because the request with the middle encoding time need not have the middle retrieval time. We must measure each request’s total before taking its median.

These timings apply to real Nomic text queries. Since the constructed strong-coordinate vectors do not come from this encoder, adding its time to their search measurements would not give us a measured text-to-results pipeline.

5 Memory limits and larger collections

5.1 Put the stages back together

To account for one complete request, let T_{shared} include query embedding, normalization, routing, and any common filtering. If T_a^{local} is the local cost for A, B, or C derived earlier, including its score table, we have

$$T_a^{\text{request}} = T_{\text{shared}} + T_a^{\text{local}} + T_{\text{final}}, \quad (14)$$

where T_{final} contains output work and any enabled reranking. All of these stages belong to the same request, so the equation does not justify adding unrelated stage medians or adding the real encoder’s measured time to constructed vectors it never produced.

For memory, we need to keep stored allocation separate from the scratch space live during a query. Write M_0 for allocated document rows, routing state, and common containers, and let D_C be C’s allocated directory, Q_B B’s extra masks and queue, and Q_C C’s extra key queue. The common query scratch remains Q_0 .

Method	Stored index allocation	Query scratch
A	M_0	Q_0
B	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
C	$M_0 + D_C$	$Q_0 + Q_C$

Actual common allocation can differ with container capacities. When applying a process RAM limit, we add the runtime and other resident data once, taking care not to add overlapping process and GPU accounting blindly. A build-memory limit must also include the temporary input and output copies.

5.2 Start with the memory calculation at one billion rows

At $N = 10^9$, $d = 256$, and eight-byte IDs, packed codes occupy 32 GB and IDs occupy 8 GB, so these two arrays alone exceed a 32 GB budget. B also keeps a second sign representation, adding at least 32 GB before per-cluster padding. Even if we change the number of probes, that stored payload remains.

Layout at 1B rows	Payload floor	Planned query RAM	64 GB budget
A, direct prefix route	40 GB	about 49 GB	Modeled fit
C, global 13-bit key	40 GB	about 49 GB	Modeled fit
B, global cluster	72 GB	about 83 GB	Rejected
B, prefix route, large leaf	72 GB	about 81 GB	Rejected

These planning values include capacity and container allowances and a configurable 1 GB runtime reserve, which is not measured runtime memory. In their current in-memory layouts, all three fail 32 GB. All listed planning cases fit 1 TB, but a machine with 1 TB need not have this machine’s memory performance. If we compressed the IDs or used another storage architecture, we would need to calculate memory again; relabeling the same index would not change the result.

5.3 When a fixed prefix carries the useful information

If we keep thirteen strong coordinates fixed under the earlier independent strong-coordinate model, the expected ideal group contains $10^9/2^{13} \approx 122,070$ rows, far more than the requested 100. The distributional calculation controls the small probability that the group is insufficient, but makes no claim about ordinary text embeddings.

A can route directly to this group and scan it, while C can look up its key and score the same rows. Both cost roughly 3 ms in the calibrated conditional model. B can also take this route with a large leaf, giving a similar time with more stored data. We should not force B through a global billion-row bitmap just to make C appear better. In this case, A and C are closely related ways to obtain the same candidate set, and the model cannot resolve a few-percent gap.

5.4 When the important coordinates change with each query

If the important coordinates change with each query, a fixed prefix may contain little useful signal, while B can choose its split coordinates from the query. We preserve the constructed path by keeping thirteen useful cuts and scaling the leaf size to 160,000. Its last-split mask payload is approximately

$$(13 + 2) \lceil 10^9/64 \rceil \times 8 = 1.875 \text{ GB}.$$

Here, the factor 15 counts the path and alternative masks live at the same time, including parent/child overlap. That count is for this prescribed path; other search histories can need different scratch.

The complete native size model predicts about 0.10 s for this global B path and about 25 s for a global scan. If we double B’s variable cost, we get about 0.20 s, which shows sensitivity to that cost rather than a confidence interval. No full billion-document query was measured.

We also need to give A the benefit of routing in this comparison. If it scores fraction f of the collection, its approximate cost is $T_{\text{route}} + fNc_{\text{scan}}$. Even with routing time ignored, A must score less than about 0.41% of the corpus to undercut the 0.10 s B prediction, while still meeting the recall target at that fraction. Scanning can win if routing cheaply

produces a small, accurate candidate set; if it cannot, the query-dependent B path is worth testing.

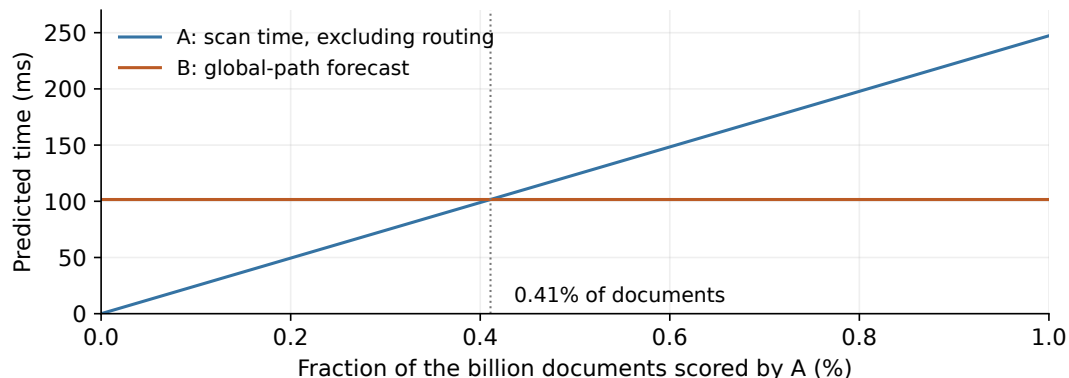


Figure 6: A conditional comparison with sufficient RAM. B is one prescribed operating point. A gets free routing in this line but must still attain the required recall at its scanned fraction. These are forecasts, not measured billion-document timings.

For C, the short key has to capture important coordinates. In the constructed changing case, a fixed 13-coordinate key misses all thirteen strong coordinates with probability about 49.94%. Even a nonempty directory lookup can therefore leave almost the entire scoring task, so key occupancy alone cannot establish an advantage in latency or recall.

5.5 What we can conclude

For the real embeddings measured here, the scan remains the supported practical baseline. The constructed comparisons show how that result depends on the workload: with a strong fixed-prefix condition, direct routing and key lookup both avoid most scores; with changing strong coordinates, B’s query-dependent splits produced a large measured advantage. Each conclusion applies to its stated workload and tested parameter space.

We can use the formulas to explain these outcomes and suggest settings, but they do not prove that every possible IVF layout loses, that Matryoshka creates the constructed distribution, or that a local timing coefficient remains exact at a billion rows. To test the favorable case on a real representation, the next useful experiment would check for the required score concentration and routing behavior while preserving the same recall and RAM constraints.

6 From the explanation to the code

If we want to follow a calculation into the implementation, the table below gives the starting points in the companion repository. Paths are relative to its root. The full implementations, saved measurements, and longer derivations are there too: the 72-page report remains in `reports/search-math/`, and this edition is in `reports/search-math-concise/`.

Question	Start here
Follow the six-document example	<code>examples/small_search.py</code>
Inspect scan and branching	<code>cpp/index.cpp</code> , <code>cpp/score.hpp</code>
Inspect key lookup	<code>cpp/key_index.cpp</code>
Check storage and work formulas	<code>experiments/models.py</code> , <code>experiments/check_controlled.py</code>
Read the final timing checks	<code>experiments/check_native_model.py</code> , <code>reports/search-math/evidence/final-study.json</code>
Calculate the larger cases	<code>experiments/project_costs.py</code> , <code>results/projected-cases-2026-09-11/</code>
Reproduce the study	<code>experiments/README.md</code> , <code>results/README.md</code>

Once the project is installed using its root README, we can run the six-document example without downloading a corpus. The second command checks the long report’s embedded arithmetic and saved evidence. The third uses frozen inputs to recalculate the larger scenarios, so it doesn’t build a billion-row index.

Listing 4: Three existing entry points, run from the repository root.

```
.venv/bin/python examples/small_search.py
python3 reports/search-math/verify_math.py
.venv/bin/python -m experiments.project_costs \
  --inputs results/projected-cases-2026-09-11/results.json \
  --output results/my-scenarios \
  --documents 1000000000 --ram-gb 32 64 1000
```

For the exact saved-input invocation and full-data runs, use the reproduction README. A full rerun also needs the dataset, embedding cache, and documented environment. Compiling this PDF only rebuilds the explanation: its tables summarize saved results, not new measurements.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024.
<https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022.
<https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources.
<https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation.
<https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Local experimental record, 10–11 September 2026. Source checkpoint `f16ca66`; `reports/search-math/evidence/` and the result files it identifies. The concise edition’s README records the selected sources and reproduction scope.